

Contrato Operativo y de Protocolo — Orden de Servicio

CUY6142 — Telepresencia y Entornos Innovadores de Colaboración Humana

1. Información del Documento

CAMPO	VALOR
Documento	Contrato Operativo y de Protocolo — Orden de Servicio (API Demo eTOM/SOM)
Versión	1.2
Fecha	14/09/2026
Preparado por	Pablo Moscoso
Curso	CUY6142 — Telepresencia y Entornos Innovadores de Colaboración Humana
Actividad relacionada	EA2 — Fundamentos y Consumo de APIs (complementa 2.3/2.4)
Documento asociado	Contrato de Datos — Orden de Servicio

Gestión de Versiones

VERSIÓN	FECHA	PREPARADO POR	SECCIÓN Y TEXTO REVISADO
1.0	14/09/2026	Pablo Moscoso	Versión inicial
1.1	14/09/2026	Pablo Moscoso	Validación de conformidad contra <code>Estandar_Documentacion_API_CUY6142.md</code> (Sección 3, checklist de Ficha de Contrato); agregada Sección 10 "Control de Versiones del Contrato", ausente respecto a la Sección 1.5 del estándar — Glosario reenumerado de 10 a 11

VERSIÓN	FECHA	PREPARADO POR	SECCIÓN Y TEXTO REVISADO
1.2	14/09/2026	Pablo Moscoso	Agregado método <code>PUT /ordenes/{id}</code> (reemplazo completo de campos editables) a la Sección 5, con nota explicativa de la diferencia con <code>PATCH</code> ; actualizadas Secciones 7 (códigos de estado) y 8 (idempotencia) en consecuencia. No se agregó "UPDATE" — no es un método HTTP válido (ver respuesta al docente)

2. Propósito y Alcance

Este documento define **cómo** un cliente interactúa con el servicio: transporte, autenticación, endpoints, formato de mensajes y códigos de estado. La estructura de los datos que viajan en cada mensaje (los campos de la Orden de Servicio) se define por separado en el **Contrato de Datos — Orden de Servicio**; este documento no repite esa definición.

El servicio simplifica el proceso eTOM **Order Handling** (Process Identifier 1.1.1.5), inspirado en el tipo de interacción cliente-servidor de un sistema real de Service Order Management (SOM), y alineado conceptualmente con el API abierto TM Forum **TMF641**, sin pretender conformidad ni certificación con ese estándar.

3. Modos de Despliegue y Host

El servicio opera en dos perfiles alternativos. Ambos usan el mismo puerto y el mismo contrato — solo cambia el host.

PERFIL	HOST	PUERTO	CUÁNDO SE USA
Demo en vivo	<IP_DOCENTE>	8081	Clase presencial. El docente comparte conexión a internet vía hotspot desde su teléfono; todos los dispositivos quedan en la misma red. El valor real de <IP_DOCENTE> se anuncia al inicio de la sesión — cambia en cada conexión al hotspot y no se documenta como valor fijo.
Distribuido	localhost (o 127.0.0.1)	8081	Cada estudiante despliega su propia instancia del servicio y prueba contra su propio equipo.

Base URL: `http://<HOST>:8081/api/v1`

Transporte: HTTP sin TLS. Es una limitación declarada, no una omisión: evita que cada estudiante deba generar y confiar un certificado autofirmado en el modo distribuido, lo que introduciría gestión de PKI ajena al objetivo del ejercicio. Consecuencia explícita: el header de autenticación (Sección 4) viaja en texto plano. Aceptable en el contexto de este ejercicio (red controlada en ambos perfiles), no aceptable como patrón para un despliegue en producción.

Fuera del alcance de este contrato: configuración de firewall del equipo servidor, límites de dispositivos conectados al punto de acceso, y aislamiento de clientes en la red utilizada. Son responsabilidad operativa de quien despliega el servicio (docente o estudiante) en el momento de uso, no una condición que el contrato de la API deba garantizar o documentar.

4. Autenticación

ASPECTO	DEFINICIÓN
Mecanismo	Header HTTP <code>X-API-Key</code>
Alcance de la clave	Única y compartida para todo el curso — no se emiten claves individuales por estudiante
Valor documental	<code><API_KEY_CUY6142></code> — el valor real lo entrega el docente el día de uso
Validez	La misma clave funciona en ambos perfiles de despliegue (demo en vivo y distribuido)
Momento de validación	Antes de procesar el cuerpo de la solicitud — es decir, antes de aplicar el Contrato de Datos
Falla	Header ausente o valor incorrecto → <code>401 Unauthorized</code> (Sección 7)

Distinción conceptual relevante para la clase: `X-API-Key` valida **quién llama** (nivel de protocolo/transporte); `cliente_id` en el cuerpo de la Orden de Servicio identifica **a nombre de quién se realiza la operación** (nivel de negocio, definido en el Contrato de Datos). Son dos conceptos independientes que en integraciones reales suelen confundirse.

5. Endpoints y Métodos

MÉTODO	RUTA	ACCIÓN	ÉXITO	ERRORES POSIBLES
POST	<code>/ordenes</code>	Crear orden	<code>201 Created</code>	<code>400</code> , <code>401</code> , <code>422</code>

MÉTODO	RUTA	ACCIÓN	ÉXITO	ERRORES POSIBLES
GET	/ordenes	Listar órdenes	200 OK	401
GET	/ordenes/{id}	Consultar una orden	200 OK	401, 404
PUT	/ordenes/{id}	Reemplazar completamente los campos editables de la orden	200 OK	400, 401, 404, 409, 422
PATCH	/ordenes/{id}	Transicionar estado	200 OK	401, 404, 409, 422
DELETE	/ordenes/{id}	Cancelar orden	200 OK	401, 404, 409

Diferencia entre PUT y PATCH en este contrato

Es la distinción central que se espera que el estudiante entienda de estos dos métodos:

- **PUT** reemplaza **toda** la representación editable de la orden (`cliente_id`, `tipo_servicio`, `prioridad`, `descripcion`). El cliente debe enviar los cuatro campos, incluso los que no cambian — un campo editable omitido se trata como ausente, igual que en una creación (Contrato de Datos, Sección 4). **PUT** es **idempotente**: enviar el mismo cuerpo dos veces deja la orden en el mismo estado final.
- **PATCH** en este contrato está deliberadamente acotado a una sola operación: transicionar `estado` según la máquina de estados del Contrato de Datos (Sección 5 de ese documento). No admite modificar `cliente_id`, `tipo_servicio`, `prioridad` ni `descripcion` — para eso se usa **PUT**.
- Ninguno de los dos métodos modifica `id`, `fecha_creacion` — son de solo lectura en todo momento. **PUT** tampoco modifica `estado`: si el cuerpo de un **PUT** incluye `estado`, el servidor lo ignora (Contrato de Datos, Sección 4).
- **PUT** solo se acepta si la orden está en `RECIBIDA` o `EN_PROGRESO`. Sobre una orden en `COMPLETADA` o `CANCELADA` responde `409 Conflict` — un estado terminal no admite edición de sus datos, la misma lógica que ya impide transicionar su `estado`.

Ejemplos de uso

Crear una orden:

```
curl -X POST http://<HOST>:8081/api/v1/ordenes \
  -H "X-API-Key: <API_KEY_CUY6142>" \
  -H "Content-Type: application/json" \
  -d '{"cliente_id":"CL-10457","tipo_servicio":"INTERNET","prioridad":"ALTA"}'
```

Consultar estado de una orden:

```
curl http://<HOST>:8081/api/v1/ordenes/<ID_ORDEN> \
  -H "X-API-Key: <API_KEY_CUY6142>"
```

Reemplazar completamente los campos editables de una orden (PUT):

```
curl -X PUT http://<HOST>:8081/api/v1/ordenes/<ID_ORDEN> \
  -H "X-API-Key: <API_KEY_CUY6142>" \
  -H "Content-Type: application/json" \
  -d '{"cliente_id":"CL-10457","tipo_servicio":"INTERNET","prioridad":"MEDIA","descripcion":"Cambio de prioridad tras contacto con el cliente"}'
```

Transicionar una orden a EN_PROGRESO:

```
curl -X PATCH http://<HOST>:8081/api/v1/ordenes/<ID_ORDEN> \
  -H "X-API-Key: <API_KEY_CUY6142>" \
  -H "Content-Type: application/json" \
  -d '{"estado":"EN_PROGRESO"}'
```

Estos mismos flujos son directamente reproducibles en Postman: header `X-API-Key`, cuerpo JSON en la pestaña Body → raw → JSON.

6. Formato de Mensajes

- `Content-Type: application/json` obligatorio en toda solicitud con cuerpo (POST, PATCH).
- Codificación: UTF-8.

- Formato de error estandarizado, idéntico en todos los endpoints:

```
{
  "error": {
    "codigo": "TRANSICION_INVALIDA",
    "mensaje": "No se puede pasar de COMPLETADA a EN_PROGRESO"
  }
}
```

7. Códigos de Estado HTTP del Contrato

CÓDIGO	SIGNIFICADO EN ESTE CONTRATO	CUÁNDO SE PRODUCE
200 OK	Operación exitosa sobre un recurso existente	GET, PUT, PATCH, DELETE exitosos
201 Created	Recurso creado	POST exitoso
400 Bad Request	El cuerpo de la solicitud no es JSON válido	Error de parseo, no de contenido
401 Unauthorized	Header <code>X-API-Key</code> ausente o incorrecto	Falla de autenticación, antes de leer el cuerpo
404 Not Found	El <code>id</code> de orden no existe	GET, PUT, PATCH, DELETE sobre <code>id</code> inexistente
409 Conflict	Operación rechazada por el estado actual de la orden: transición de <code>estado</code> no permitida por la máquina de estados, o intento de <code>PUT/DELETE</code> sobre una orden en estado terminal (<code>COMPLETADA/CANCELADA</code>)	PUT, PATCH o DELETE que viola la Sección 5 del Contrato de Datos
422 Unprocessable Entity	JSON válido pero viola el Contrato de Datos (campo obligatorio ausente, valor fuera de enum)	POST, PUT o PATCH con datos que no cumplen el schema
500 Internal Server Error	Fallo no controlado del servidor	Excepción no anticipada

Distinción 400 vs 422: 400 significa que el cuerpo ni siquiera es JSON parseable; 422 significa que es JSON válido pero no cumple las reglas del Contrato de Datos (Sección 4 de ese documento). Son dos capas de validación distintas y se comunican con códigos distintos.

8. Idempotencia y Concurrency

- GET, PUT, DELETE y PATCH hacia un estado ya alcanzado son idempotentes: repetir la llamada con el mismo cuerpo dado dejan la orden en el mismo estado final. Matiz: `fecha_actualizacion` cambia en cada llamada exitosa aunque el resto de los campos no varíe — es un efecto secundario esperado, no invalida la idempotencia del recurso en sí.
 - POST no es idempotente: cada llamada crea una nueva orden, incluso con el mismo cuerpo.
 - No hay control de concurrencia (sin bloqueo optimista ni versión de recurso). Es una limitación declarada, aceptable para un servicio de un solo proceso en memoria usado en clase, no para un entorno multiusuario real.
-

9. Exclusiones Declaradas del Contrato

- Persistencia real: almacenamiento en memoria, se reinicia al reiniciar el proceso.
 - TLS/HTTPS (ver Sección 3).
 - Autenticación individual por estudiante (Sección 4).
 - Configuración de red del entorno de ejecución: firewall, límites del punto de acceso, aislamiento de clientes (Sección 3) — responsabilidad operativa, no parte de este contrato.
-

10. Control de Versiones del Contrato

- Agregar un endpoint nuevo, o un código de estado adicional que no cambie el comportamiento de los existentes: cambio menor, no rompe clientes existentes.
- Cambiar el método HTTP de un endpoint existente, eliminar un endpoint, modificar el mecanismo de autenticación, o cambiar el significado de un código de estado ya definido: cambio mayor — rompe clientes existentes, requiere una nueva versión de la base URL (ej. `/api/v2`) y coordinación con el Contrato de Datos si el cambio también afecta la representación del recurso.
- Esta ficha documenta la versión 1.0 del contrato operativo, correspondiente a la base URL `/api/v1`.

11. Glosario

TÉRMINO	DEFINICIÓN
Contrato Operativo	Documento que define cómo se accede a un servicio: transporte, autenticación, endpoints y manejo de errores.
Protocolo	Conjunto de reglas que rigen el intercambio de mensajes entre cliente y servidor (en este caso, HTTP + JSON).
Idempotencia	Propiedad de una operación cuyo resultado no cambia si se repite más de una vez con los mismos parámetros.
Header HTTP	Metadato enviado junto a una solicitud u respuesta HTTP, fuera del cuerpo del mensaje.
Status Code	Código numérico HTTP que indica el resultado de una solicitud.

Documento de referencia — no es un procedimiento (MOP). Se actualiza cuando cambia el protocolo, la autenticación o los endpoints del servicio; los cambios en la estructura de datos se documentan en la ficha de Contrato de Datos asociada.